



FACULDADE SENAC-DF

Lógica de Programação com Python

Lista de Exercícios — Aula 1

Conceitos de Lógica de Programação e Pensamento Computacional

Estes exercícios acompanham a Aula 1 e têm como objetivo praticar os primeiros passos em Python: comandos, comentários, variáveis, entrada e saída de dados, e manipulação básica de strings. Siga a ordem sugerida — os exercícios seguem uma progressão de dificuldade.

Parte 1 — Primeiros Passos em Python

Nesta parte você vai praticar o comando `print()`, o uso de comentários e a criação de variáveis — a base de qualquer programa em Python.

Exercício 1 • Olá, Mundo! (exercício-modelo)

Objetivo: Compreender a estrutura mínima de um programa em Python e o uso de comentários.

Todo programa começa por algo simples. O exemplo abaixo já está resolvido — use-o como modelo para os próximos exercícios.

```
# Este é o meu primeiro programa em Python
# O símbolo # inicia um comentário: o Python não executa essa linha,
# ela serve apenas para explicar o código para quem está lendo.

# A função print() exibe uma mensagem na tela
print("Olá, Mundo!")
```

Atividades:

- Digite e execute o código acima em seu ambiente Python.
- Altere a mensagem para exibir o seu próprio nome, por exemplo: Olá, meu nome é
- Adicione um comentário acima do `print` explicando o que aquela linha faz.

Exercício 2 • Sequência de números

Objetivo: Praticar a ideia de estrutura sequencial: uma lista de instruções executadas uma após a outra, na ordem em que aparecem.

Escreva um programa que imprima os números de 1 até 10, um em cada linha.

Dica: por enquanto, use um comando `print()` para cada número (assim como fizemos no Exercício 1). Mais adiante na disciplina veremos como repetir comandos automaticamente com estruturas de repetição (laços).

Exercício 3 • Trabalhando com variáveis

Objetivo: Entender o que é uma variável e como armazenar e exibir valores.

Uma variável funciona como uma caixa onde guardamos um valor para usar depois. Veja o exemplo:

```
# Atribuição de variável
numero1 = 5

# Imprimir o valor armazenado em 'numero1'
print(numero1)
```

Atividades:

- Crie uma variável chamada nome e armazene o seu nome nela. Em seguida, imprima o valor.
- Crie uma variável chamada idade e armazene a sua idade. Imprima o valor.
- Crie duas variáveis, cidade e curso, e imprima as três variáveis (nome, cidade e curso) em três linhas separadas.

Exercício 4 • Soma de dois números

Objetivo: Aplicar o algoritmo visto em aula — Leia dois números → Some os dois números → Mostre o resultado — em código Python.

Escreva um programa que:

- Peça ao usuário para digitar dois números (use a função `input()`).
- Converta os valores digitados para número (`int()` ou `float()`).
- Calcule a soma dos dois números.
- Mostre o resultado na tela, com uma mensagem clara (ex.: "A soma é: 15").

Dica: `input()` sempre retorna um texto (string) — por isso é necessário converter o valor para número antes de somar, usando `int(valor)` ou `float(valor)`.

Exercício 5 • Operadores Aritméticos Básicos

Objetivo: Conhecer e praticar os operadores aritméticos básicos do Python: adição (+), subtração (-), multiplicação (*) e divisão (/).

O Python utiliza símbolos (operadores) para realizar cálculos matemáticos entre números. Veja o exemplo abaixo:

```
a = 10
b = 3

# Adição
print(a + b)    # -> 13

# Subtração
print(a - b)    # -> 7

# Multiplicação
print(a * b)    # -> 30

# Divisão
print(a / b)    # -> 3.3333333333333335
```

Atividades:

- Crie duas variáveis numéricas, num1 e num2, e atribua a elas os valores 8 e 4.
- Calcule e imprima o resultado da adição (num1 + num2).
- Calcule e imprima o resultado da subtração (num1 - num2).
- Calcule e imprima o resultado da multiplicação (num1 * num2).
- Calcule e imprima o resultado da divisão (num1 / num2).
- Para cada operação, imprima uma mensagem clara indicando qual cálculo foi feito, por exemplo: "A soma é: 12".

Dica: em Python, a divisão (/) sempre retorna um valor decimal (float), mesmo que o resultado seja exato. Existem também os operadores de divisão inteira (//) e resto da divisão (%), que veremos mais adiante.

Parte 2 — Strings: Índices e Fatiamento (Slicing)

Uma string é uma sequência de caracteres. Podemos acessar cada caractere individualmente através da sua posição (índice), e também extrair "pedaços" dela através do fatiamento (slicing).

Indexação:

- Use colchetes [] para acessar um caractere pela sua posição (índice).
- A indexação em Python começa em 0: o primeiro caractere está no índice 0, o segundo no índice 1, e assim por diante.
- Índices negativos contam a partir do final: -1 é o último caractere, -2 é o penúltimo, e assim por diante.

Fatiamento (Slicing):

- A sintaxe [início:fim:passo] permite extrair uma parte de uma string (ou de outra sequência).
- [::] copia a sequência inteira.
- Um número após o segundo dois-pontos, como [::2], define o intervalo entre os elementos selecionados.
- [::-1] inverte a sequência.

```
texto = "Python"

print(texto[0])      # primeiro caractere -> P
print(texto[-1])     # último caractere    -> n
print(texto[0:3])    # do índice 0 ao 2    -> Pyt
print(texto[::-1])   # string invertida    -> nohtyP
```

Exercício 6 • Primeiro caractere

Dada a string: Python, imprima o primeiro caractere.

Exercício 7 • Localizando um caractere

Dada a string: Hello, World!, imprima o caractere "W".

Exercício 8 • Fatiando o início da string

Dada a string: Data Science, imprima os três primeiros caracteres.

Exercício 9 • Fatiando o final da string

Dada a string: Machine Learning, imprima os três últimos caracteres.

Exercício 10 • Desafio bônus

Objetivo: Combinar variáveis, entrada de dados e fatiamento de strings em um único programa.

Peça ao usuário para digitar o seu nome completo. Em seguida:

- Imprima apenas a primeira letra do nome digitado.
- Imprima o nome digitado ao contrário (invertido).
- Imprima quantos caracteres o nome possui (dica: pesquise sobre a função `len()`).

situações que veremos em breve.